LANCE and IoTChainBench: A Network-Native LLM Harness and Benchmark for Multi-Hop IoT Penetration Testing

Anonymous Authors
Paper #XXXX, Anonymized for Double-Blind Review

***Abstract*—LLM penetration-testing agents are typically evaluated on isolated hosts, while IoT compromises often depend on reachability across gateways, brokers, and cameras. This leaves a methodological gap: single-host corpora do not measure pivots, protocol roles, or evidence depth.**

We present IoTChainBench, a reproducible network-scale IoT benchmark composed of 13 Proxmox/Ansible scenarios. The benchmark provides per-vulnerability ground truth and structured metadata for evaluating findings across devices and network paths. We also introduce LANCE (LLM Agent for Network Compromise Evaluation), a six-phase network-native harness that combines topology analysis, active discovery, triage, verification, intrusion attempts, and report generation.

In a controlled comparison under the same model and evaluator, LANCE achieves 0.959 recall, 0.914 precision, 0.935 F1, and 86.4% CVSS-weighted score on the 12 evaluated scenarios, outperforming every tested LLM agent harness in every scenario. On external single-host benchmarks used to assess behavior outside the network setting, LANCE reaches 27.3% end-to-end success under the standard AutoPenBench setting and 30.2% useful findings on a 328-case Vulhub run.

***Index Terms*—IoT security, penetration testing, large language models, autonomous agents, benchmark, cyber-physical systems**

1. Introduction

IoT deployments are compromised through paths, not just devices. Mirai [1], Verkada [2], and Stuxnet [3] each demonstrate this: impact came from chained access across cameras, brokers, PLCs, and segmented boundaries.

Yet LLM penetration-testing agents are evaluated on a single host at a time: isolated containers, VMs, or CTF services scored independently. This gap is methodological, not merely architectural. Single-host corpora cannot measure three capabilities central to IoT/OT security: finding vulnerabilities behind pivots, reasoning over domain protocols (MQTT, Modbus, CoAP, LoRaWAN), and requiring proof before counting a finding. Without a network-scale benchmark, strong single-host scores overstate readiness for real deployments.

Contributions. This paper makes two main contributions.

- **IoTChainBench**: a reproducible network-scale benchmark that tests multi-hop IoT/OT compromise across gateways, brokers, cameras, sensors, and service dependencies. Each scenario is deployed via Ansible and includes per-vulnerability ground truth for evaluating agent findings.
- **LANCE**: the LLM Agent for Network Compromise Evaluation, a six-phase network-native harness (topology $\rightarrow$ discovery $\rightarrow$ triage $\rightarrow$ verification $\rightarrow$ intrusion $\rightarrow$ report) for multi-hop IoT penetration testing. LANCE requires evidence before counting a vulnerability as found and achieves 0.935 F1 on the 12 evaluated scenarios, while recovering 86.4% of the vulnerability severity mass.

All artifacts are released at `ANONYMIZED_URL`.

2. Related Work

This section reviews prior work relevant to our approach. We first survey LLM-driven penetration-testing agents and the benchmarks used to evaluate them, then discuss the classical attack-graph literature that informs LANCE's topology-aware design, and finally review IoT firmware analysis and the security frameworks underlying our scenario taxonomy.

2.1. LLM Penetration-Testing Agents

Single-agent systems. Happe and Cito [4] were among the first to show that a single LLM driving a shell can autonomously perform privilege escalation on Linux VMs. PENTESTGPT [5] improved this with a three-module design over a *Pentesting Task Tree*, outperforming direct GPT-3.5 and GPT-4 usage on HackTheBox and VulnHub targets. HACKINGBUDDYGPT [6], AUTOATTACKER [7], and NEBULA [8] extend this line as open-source tools.

Multi-agent pipelines. VULNBOT [9] decomposes pentesting into reconnaissance, scanning, and exploitation over a *Penetration Task Graph*, gaining 21 points end-to-end over its single-agent baseline (30.3% on AutoPenBench with Llama 3.1-405B). AUTOPENTESTER [10] adds RAG across five agents for a $+27\%$ subtask gain over PENTESTGPT on HackTheBox. PENTESTAGENT [11] and CAI [12] generalise this blueprint with modular tool use and a public leaderboard.

Model-level improvements. PENTEST-R1 [13] applies two-stage reinforcement learning on reasoning traces. XOFFENSE [14] fine-tunes Qwen3-32B on Chain-of-Thought penetration testing data, reaching 72.72% task completion and 79.17% best-of-five subtask completion on AutoPenBench. CHECKMATE [15] couples an LLM with a *Classical Planning+* module, reaching an interactive shell (milestone M7) on 88% of 120 Vulhub targets.

Common limitation. Despite this diversity, reported evaluations remain single-target or challenge-level. They do not score subnet reachability, pivot depth, or per-vulnerability progress across IoT/OT protocols (MQTT, Modbus, LoRaWAN, CoAP), which is the gap IOTCHAINBENCH and LANCE address.

2.2. LLM Cybersecurity Benchmarks

Challenge-level corpora. AUTOPENBENCH [16] is the dominant evaluation substrate: 33 Docker-container tasks with milestone and flag-based scoring, used by VULNBOT, PENTEST-R1, and XOFFENSE. VULHUB [17] provides reproducible vulnerable environments used by CHECKMATE. CAIBENCH [18], NYU CTF BENCH [19], and CYBENCH [20] extend coverage to CTFs, attack-and-defense ranges, and knowledge tests. These corpora score at challenge or flag level. They do not provide per-vulnerability ground truth tied to reachability, hop depth, or IoT/OT protocols.

Multi-host labs. LUDUS [21] and GOAD [22] deliver Proxmox+Ansible deployments of enterprise Active Directory networks. They are labs and training ranges, not scored LLM benchmarks. Both model multi-host compromise and lateral movement, but neither ships a per-vulnerability ground-truth schema, hop-depth annotations, or an evaluator. Both also target enterprise AD rather than IoT/OT protocols. To our knowledge, no prior benchmark combines multi-host reachability, IoT/OT protocol coverage, per-vulnerability ground truth, and evidence-level scoring beyond binary flag capture (Table 1).

2.3. Attack Graphs and Network Reachability

Representing a network as a graph for security analysis predates deep learning by decades. Phillips and Swiler [23] introduced graph-based network-vulnerability analysis. Sheyner *et al.* [24] developed automated attack-graph generation via model checking, and MULVAL [25] cast the problem as Datalog inference. These systems compute reachable attacker states from a *static* encoding of host configurations and exposed services. LANCE borrows this reachability abstraction but operates on a *live* network: edges are validated by probes, the LLM proposes the next action, and the graph is updated as hosts are discovered. Classical attack-graph generation is therefore complementary rather than competing.

2.4. IoT Security Context

Scenario design and ground-truth taxonomy follow three references: OWASP IoT Top 10 [26] for vulnerability categories, MITRE ATT&CK for ICS [27] for OT-tactic annotations, and NIST SP 800-82 Rev. 3 [28] for OT security and segmentation conventions.

Empirical IoT-security artefacts fall into two families. **Network-traffic datasets** (IoT-23 [29], TON_IoT [30], Edge-IIoTset [31]) provide labeled traces for intrusion detection. They are not deployable pentest targets with per-vulnerability ground truth or controllable reachability. **Firmware-focused work** (Costin *et al.* [32], FIRMADYNE [33], FIRMAE [34], IOTGOAT [35]) exercises single-device firmware analysis. These datasets measure detection or firmware-analysis capability. IOTCHAINBENCH instead measures active security assessment on deployed multi-device networks with firewall-enforced reachability.

TABLE 1. GAP ANALYSIS ACROSS LLM PENTEST AGENTS AND CORPORA.

System	Scale	IoT/OT	GT	MH
LLM pentest agents				
Happe & Cito [4]	1 host	✗	flag	✗
PentestGPT [5]	1 host	✗	task-step	✗
VulnBot [9]	1 host	✗	task-step	✗
AutoPentester [10]	1 host	✗	task-step	✗
CAI [12]	1 host	✗	flag	✗
Pentest-R1 [13]	1 host	✗	task-step	✗
xOffense [14]	1 host	✗	task-step	✗
EnIGMA [36]	1 host	✗	flag	✗
CheckMate [15]	1 host	✗	task-step	✗
Evaluation corpora				
AutoPenBench [16]	1 host	✗	flag	✗
CAIBench / RCTF2 [18]	1 host	partial	flag	✗
Vulhub [17]	1 host	✗	none	✗
Ludus + GOAD [21], [22]	network	✗	none	✗
IoTChainBench (ours)	**network**	✓	**per-vuln**	✓

3. Threat Model and Problem Definition

Problem statement. Given a target subnet with a designated entry point e declared in each scenario's topology metadata, *network-scale penetration testing* is the task of discovering all hosts reachable from e, identifying their vulnerabilities, producing evidence of exploitability, and reporting findings across the full network. The unit of evaluation is the network, not an individual host: IoT risk is driven by pivot chains (e.g., entry point $\rightarrow$ gateway $\rightarrow$ MQTT broker $\rightarrow$ OT controller), and single-host metrics are blind to vulnerabilities that only become reachable after one or more pivots.

Attacker model. The attacker is an external adversary with Layer-3 access to the target subnet but no prior credentials, no out-of-band knowledge of deployed software, and no privileged vantage beyond active scanning. Their objective is data exfiltration (sensor readings, credentials) or OT disruption (reaching industrial-control services behind

a segmentation boundary). They may use standard pentest tooling (Nmap, MQTT clients, SSH, HTTP) augmented by an LLM with tool-calling, within a bounded turn budget (§5.2). This adversarial setting requires multi-step reasoning across heterogeneous protocols, which motivates the use of an LLM-driven agent rather than a static scanner.

Evidence levels. A finding is solved at the deepest level the agent can demonstrate within its turn budget:

- `L1` *detection*: a port scan or banner confirms the issue (e.g., unauthenticated MQTT port `1883` open).
- `L2` *exploitation*: an active interaction demonstrates it (e.g., anonymous MQTT subscribe on `#`, SSH login with default credentials).
- `L3` *exfiltration*: sensitive content is retrieved (e.g., MariaDB `root` dump, camera RTSP stream).

This ordering governs both LANCE Phase 4 prompting (§5.1) and evaluator scoring (§4.5).

Scope. We evaluate at the IP/application layer on virtual IoT networks. Out of scope: physical and side-channel attacks, RF-layer exploitation, firmware extraction, post-exploitation persistence, and insider threats.

4. IoTChainBench: A Network-Scale Benchmark

4.1. Design Principles

IoTChainBench is built around six principles:

- **Reproducibility.** Full infrastructure deploys from Ansible playbooks against a Proxmox cluster; vulnerabilities are injected idempotently with no manual steps.
- **Architectural diversity.** Scenarios span five network patterns with reachability enforced by OpenWrt firewall rules in segmented/VLAN cases.
- **Protocol coverage.** Each scenario mixes IoT and OT protocols (MQTT, Modbus, LoRaWAN, CoAP, SNMP, SSH, HTTP, FTP, Telnet) to exercise protocol-specific reasoning.
- **Ground-truth granularity.** Every vulnerability is documented in YAML with device, IP, severity, OWASP IoT and MITRE ICS mappings, optional CVE, indicators, and a verification command.
- **Scalability axis.** Ten main scenarios (4–8 devices) and two scalability scenarios (15 and 35 devices) separate pattern coverage from network size.

4.2. Network Topologies and Scenario Catalogue

IoTChainBench defines five topological patterns that vary the number of pivots required to reach the deepest vulnerable service, directly setting the pivot-depth difficulty axis:

nosep

1) **P1 — Flat.** Single subnet; all devices directly reachable from the entry point.
2) **P2 — Gateway-centric.** One IoT gateway mediates all back-end access; back-end services unreachable without gateway compromise.
3) **P3 — Segmented IT/OT.** Admin/IT/OT zones with ACLs; the OT zone (Modbus PLCs, HMI) requires a jump-host pivot.
4) **P4 — Hub-centric star.** A central hub is the sole path to the MQTT broker, DB, and camera; peripherals are mutually isolated.
5) **P5 — Edge-cloud pivot.** Edge and cloud subnets are separated; only the edge gateway bridges them.

The 12 scenarios (Table 2) are organised along two axes: 10 *main* scenarios instantiate the patterns above at 4–8 devices, and 2 *scalability* stressors push to 15 and 35 devices using larger multi-zone variants.

TABLE 2. IoTChainBench scenario catalogue.

ID	Pattern / role	Dev.	Vulns	Theme
Main scenarios (pattern coverage)				
`s_1`	P1 Flat	4	12	Minimal IoT (MQTT/Web/SSH)
`s_2`	P1 Flat	6	13	Flat IoT + IT mix
`s_3`	P2 Gateway-centric	8	18	NATO lab replica
`s_4`	P3 Segmented IT/OT	8	18	Industrial (Modbus/HMI)
`s_5`	P4 Hub-centric star	8	15	Smart building (cameras/NVR)
`s_6`	P4 Hub-centric star	6	16	Home automation hub
`s_7`	P5 Edge-cloud pivot	6	14	Edge → cloud API
`s_8`	P3 Segmented IT/OT	8	14	IT/IoT/OT 3-zone industrial
`s_9`	P1 Flat (mesh)	6	11	Mesh IoT smart-city outdoor
`s_10`	P1 Flat (variants)	6	13	Role-variant stress
Scalability stressors				
`s_11`	Smart City (flat)	15	23	3 zones same /24 (no isolation)
`s_12`	Smart City (large)	35	42	Largest network (34 LXC services)
Total		**116**	**209**	

4.3. Deterministic Vulnerability Injection

All vulnerabilities are injected by a dedicated Ansible role parameterized by `scenario_id`, ensuring fully reproducible and idempotent deployments. Each scenario YAML declares router-level weaknesses (Telnet, WAN admin, FTP) and per-service role definitions that trigger targeted injections, covering the most common real-world IoT misconfigurations: anonymous MQTT access, weak SSH credentials, and passwordless database roots.

The 209 injected vulnerabilities span 11 categories (Table 3), dominated by misconfigurations (62), data exposure (36), and missing authentication (34). Severity skews high: 95 HIGH and 33 CRITICAL instances, reflecting realistic IoT deployment weaknesses rather than theoretical edge cases.

4.4. Ground-Truth Schema

Each vulnerability-injected scenario ships a YAML file with one entry per injected vulnerability. Each entry records

TABLE 3. VULNERABILITY CATEGORY AND SEVERITY DISTRIBUTION.

Category	Count	CRIT	HIGH	MED	LOW
misconfiguration	62	7	30	25	0
no_authentication	34	14	18	2	0
data_exposure	36	0	14	22	0
default_credentials	26	3	23	0	0
info_disclosure	13	0	0	0	13
code_injection	10	9	1	0	0
cve	9	0	3	6	0
insecure_update	9	0	4	5	0
weak_crypto	5	0	0	0	5
missing_header	3	0	0	0	3
privilege_escalation	2	0	2	0	0
Total	**209**	33	95	60	21

the affected device, IP, role, severity, category, OWASP IoT Top 10 and MITRE ATT&CK for ICS mappings, an optional CVE, expected detection indicators, a verification command that confirms exploitability, and reachability metadata for pivot-aware analysis. This per-vulnerability granularity, absent from flag-capture benchmarks, supports the common matcher and severity-weighted score used in our evaluation.

```
- id: V1
  title: "MQTT without authentication"
  severity: high
  category: misconfiguration
  owasp_iot: "I1 / I9"
  mitre_ics: "Initial Access, Collection"
  indicators:
    - "allow_anonymous true"
  verification: "mosquitto_sub -h 192.168.100.11 -t '#' -v"
```

Figure 1. Example ground-truth entry.

Scenarios may additionally declare a `bonus_types` list of vulnerability types adjacent to the injected set. Agent findings that match these types are counted as neither True Positives nor False Positives, avoiding penalties for legitimate hardening recommendations while keeping the recall denominator fixed.

4.5. Scoring and Diagnostic Fields

Finding matcher. Each LLM finding is matched against ground truth in three ordered passes: (1) exact CVE match; (2) `(ip, vuln_type)` with canonicalised type; (3) `(ip, severity)` fallback. Unmatched findings are False Positives. Duplicate findings on `(ip, type, port)` are resolved by retaining the lower severity to counteract inflation observed in pilot runs.

Weighted score. The composite score is a severity-weighted True-Positive sum (w: critical 4, high 3, medium 2, low 1) normalized by $\sum_{g \in \mathrm{GT}} w(g)$, with $0.5\times$ and $0.75\times$ penalties for fallback and severity-mismatch matches respectively.

Evidence levels. Ground-truth entries and agent reports can record the deepest evidence level reached: L1 detection, L2 exploitation, and L3 exfiltration. These labels support later evidence-depth analysis, but the reported comparison in this paper aggregates findings only as True Positives, False Positives, recall, precision, F1, and severity-weighted score.

Hop-depth metadata. For each ground-truth vulnerability g, the benchmark can store the minimum number of pivots from entry point e:

$$\text{hop\_depth}(g) = \min_{p \in \text{paths}(e \to d_g)} \max(0,\, |p|_E - 1).$$

We use this metadata to design multi-hop scenarios and to interpret qualitative failure modes. We do not report a separate hop-depth aggregate in the present evaluation.

5. LANCE: A Network-Native LLM Pentest Harness

Existing LLM pentest harnesses treat the network as a flat list of targets, letting a single agent accumulate unbounded context as the network grows. LANCE instead decomposes network pentesting into six phases with typed deliverables. The key design decision is *parallelization at the device and vulnerability granularity*: rather than a single monolithic agent holding the full network state, we spawn dedicated micro-agents per target with constrained tool sets and aggregate deterministically. This keeps each micro-agent's context bounded at $O(1)$ in the number of network devices, with cross-device state re-introduced only in Phase 5 (intrusion) and Phase 6 (report). Vulnerability type strings produced by LLM agents are resolved through a canonical taxonomy of 11 categories (Table 3), filtering noise and deduplicating collisions before evaluation.

5.1. Pipeline

Phase 1: Topology prior. Phase 1 loads a directed graph $G = (V, E)$ of the target network, where V are devices and E are protocol-labeled links. In *scenario mode*, the topology is loaded from the benchmark YAML; in *discovery mode*, the graph starts empty and is populated by Phase 2. The output enumerates the attack surface and ranks pivot candidates by betweenness centrality (identifying devices whose compromise enables the most lateral paths).

Phase 2: Network discovery. Phase 2 executes active reconnaissance across the target `/24` using host discovery, service/version scanning, L2 vendor probing, and protocol-specific probes (MQTT, SSH, HTTP). Unlike single-host harnesses, the agent derives the target IP from the CIDR scope rather than receiving it directly.

Phase 3: Per-device parallel triage. Phase 3 runs a deterministic scanner pass per device, then spawns one LLM micro-agent per device with only that device's scan output and a minimal tool set (`http_get`, `cve_search`). Per-device outputs are merged and deduplicated via severity-aware rules. This decomposition keeps each agent's context constant regardless of network size, at the cost of cross-device correlation, which Phase 5 partially recovers through lateral movement.

Phase 4: Per-vulnerability verification. Phase 4 spawns one micro-agent per Phase 3 finding with full operational tool access (`ssh_login`, `mysql_query`, `mqtt_listen`, `modbus_scan`, `telnet_connect`). Each agent attempts the deepest evidence level achievable (L1–L3, §3) within a bounded turn budget. Phase 3 confirmed findings override Phase 4 errors to avoid erasing directly-observed indicators.

Phase 5: Intrusion and lateral movement. Phase 5 turns the per-device vulnerability inventory into a network-scoped infiltration campaign. The intrusion agent attempts credential spraying, lateral movement along edges present in the topology graph from Phase 1 (reflecting observed or asserted reachability), and crown-jewel access on back-end services unreachable from the entry point. Every hop is recorded with source, destination, protocol, and credentials used, providing the evidence trail used for network-scoped reporting.

Phase 6: Network report. Phase 6 produces a network-scoped report with device inventory, per-device vulnerabilities, attack surface summary, multi-hop intrusion narrative, and recommendations ordered by severity $\times$ exposure. Cross-device aggregation is deliberately deferred to this phase to avoid bloating earlier agents, and the consolidated output enables a human analyst to prioritize remediation across the full network rather than per-host.

5.2. Cost Envelope

For a network with n devices and v findings per device on average, the harness issues $O(nv)$ LLM invocations:

$$\underbrace{1}_{P1} + \underbrace{1}_{P2} + \underbrace{n}_{P3} + \underbrace{nv}_{P4} + \underbrace{1}_{P5} + \underbrace{1}_{P6}$$

Each invocation is bounded by a fixed turn budget (50 for P1–P2, 10 per device for P3, 10 per finding for P4, 30 for P5, 20 for P6). Cost scales linearly with network size, not combinatorially.

The three structural choices above (topology prior P1, per-device parallelism P3–P4, and the dedicated intrusion phase P5) define the architecture evaluated in §7.

6. Experimental Setup

Infrastructure. All scenarios are deployed on a single Proxmox VE host as LXC containers on a dedicated Linux bridge with an OpenWrt router as gateway; per-scenario deployment and teardown are fully automated via Ansible.

Model. To isolate architectural contributions from model-specific advantages, all LLM-driven systems run on the same general-purpose model: **MiniMax-M2.7** [37]. Security-fine-tuned models are intentionally excluded; this study addresses the architecture axis, not the model fine-tuning axis.

Baselines. We compare LANCE against two adapted LLM-agent baselines, reimplementing published pipeline architectures on the same scenarios and model. The adapted baselines are run in the informed setting; LANCE is reported in both informed and blind modes:

- **CAI-style adapter**: single-agent tool-calling loop, adapted from the CAI [12] architecture.
- **VulnBot-style adapter**: multi-agent with a Penetration Task Graph, adapted from VulnBot [9].

These systems are designed primarily for single-host or single-target workflows, not for full-network graph reasoning. We therefore run each adapted baseline once per ground-truth target device in a scenario, normalize the per-device outputs into the common finding schema, merge them at scenario level, and score the merged output with the same evaluator as LANCE.

Metrics. We report recall (task completion), precision, F1, and CVSS-weighted score, formally defined in §4.5. Evidence levels and hop-depth metadata are retained in the artifact for diagnostic analysis, but they are not reported as aggregate metrics in this paper.

Variance. Reference runs use fixed artifacts, fixed prompts, and temperature 0 decoding. For LANCE, we additionally run a same-artifact repeat in both informed and blind modes; §7.2 reports the observed stability. The adapted baselines are reported from one controlled run per target device.

7. Evaluation

The evaluation is organized around three questions. First, does LANCE improve per-vulnerability scoring on the 12 network-scale IoT comparison scenarios? Second, is the gain consistent across topology types, rather than concentrated in one easy scenario? Third, how does the same harness transfer to established single-host corpora? Cross-corpus results are used only for context: models, success oracles, and task definitions differ across papers.

7.1. Overall performance

Table 4 is the controlled comparison on the 10 IoTChain-Bench scenarios for which all adapted baselines were run. It reports per-scenario F1 for LANCE in informed and blind modes, including a same-artifact repeat for each mode, followed by the CAI and VulnBot adapters, all using the same MiniMax-M2.7 model and evaluator. Since the comparison systems are single-target agents rather than multi-device network harnesses, we run each baseline once per target device, merge its per-device findings at scenario level, and then compute the F1 shown in the table. Over the same scenarios, informed LANCE also reaches 0.962 recall, 0.922 precision, and an 86.1% CVSS-weighted score.

The main result is that both reference LANCE modes exceed both adapted baselines on every comparison scenario. The baselines are not empty detectors—they recover part of the injected ground truth—but their coverage remains too shallow to raise F1 above 0.55 in any scenario.

7.2. Consistency across modes and reruns

The gain is not driven by a single context setting or topology. The repeat columns in Table 4 show two different stability profiles: informed LANCE changes by only

TABLE 4. PER-SCENARIO F1 ON THE 10 COMPARISON SCENARIOS. A IS THE REFERENCE RUN; B IS THE SAME-ARTIFACT REPEAT.

Scenario	Inf.A	Inf.B	Blind.A	Blind.B	CAI	VulnBot
S1	0.923	0.960	0.880	0.880	0.150	0.150
S2	0.923	0.833	0.923	0.740	0.350	0.450
S3	0.952	0.971	0.941	0.810	0.000	0.000
S4	0.944	0.788	0.909	0.565	0.360	0.440
S5	1.000	0.965	0.929	0.126	0.500	0.500
S6	1.000	0.970	1.000	0.903	0.400	0.400
S7	0.965	0.783	0.889	0.786	0.500	0.450
S8	0.933	0.963	0.923	0.815	0.550	0.350
S9	0.952	0.952	0.857	0.909	0.170	0.290
S10	0.815	0.960	0.769	0.880	0.140	0.140
Mean	0.940	0.914	0.902	0.741	0.312	0.316

0.026 mean F1, while blind LANCE drops from 0.902 to 0.741. The most extreme failure mode is structural: both adapted baselines score 0.000 on S3, a gateway-centric topology where back-end services are only reachable after pivoting through the gateway. A single-host agent has no network prior and therefore never discovers those services; LANCE's Phase 1 topology analysis makes the dependency explicit before any scan begins.

The informed repeat is not uniformly shifted. Most scenario deltas are below 0.04 F1, but S7 and S4 drop by 0.182 and 0.156 respectively, mostly through missed findings rather than new false positives; conversely, S10 improves by 0.145. The remaining larger change is S2 (-0.090). Blind mode is more volatile: the repeat loses substantial coverage on S5 and S4, but improves on S10. Thus the rerun variance is concentrated in a few topology cases, while the reference-run ordering against the adapted baselines is unchanged.

The trend also holds across topology families. LANCE is exact on the two hub-centric star topologies (S5, S6), and remains high on segmented IT/OT and edge-cloud settings (S4, S7, S8, S9). The hardest scenario is S10 (F1 = 0.815), an intentionally flat topology where service-role variants increase semantic ambiguity and duplicate reporting. The two scalability extensions (S11: 15 devices; S12: 35 devices) confirm the trend: F1 = 0.875 and F1 = 0.942 respectively.

Across the larger 13-scenario repeat batch, informed and blind reach macro-F1 0.916 and 0.643 respectively. This is not a full confidence-interval study, but it checks that the main result is not an obvious single-call artifact.

7.3. Cross-benchmark comparison on single-host corpora

To assess transfer beyond IoT multi-hop scenarios, we run LANCE and the adapted baselines on two single-host corpora used by prior work. These results stress the single-target exploitation component inside the harness; they are contextual rather than head-to-head with published papers because models, target subsets, and success oracles differ.

AutoPenBench. AutoPenBench defines 33 flag-based tasks with published E2E rates. We run *blind* mode (endpoint and service metadata only) and *informed* mode (also the case identifier and vulnerability label). Table 5 separates published paper results from our controlled MiniMax-M2.7 runs.

TABLE 5. AUTOPENBENCH TRANSFER RESULTS.

System	Model	Sub-task	E2E
Published results			
Llama baseline [9], [14]	Llama 3.1-405B	49.1	9.1
GPT-4o baseline [9], [14]	GPT-4o	—	21.2
VulnBot [9]	Llama 3.1-405B	69.1	30.3
xOffense [14]	Qwen3-32B LoRA	79.2¶	72.7
This work — same tasks, model, and oracle			
CAI adapter [12]	MiniMax-M2.7	12.1†	6.1
VulnBot adapter [9]	MiniMax-M2.7	18.2†	15.2
LANCE informed	MiniMax-M2.7	51.5†	27.3
LANCE blind	MiniMax-M2.7	39.4†	21.2

Models and run protocols differ, so this is not a controlled ranking. "—" indicates a metric not reported by the original paper. † *Our sub-task analogue is* useful findings*: tasks where the agent produced actionable exploit evidence, regardless of flag capture. Our local runs do not use the upstream AutoPenBench orchestrator.* ¶ *xOffense's sub-task score is best-of-five; its E2E score is strict task completion.*

Blind LANCE captures 7/33 flags and 13/33 useful findings; informed LANCE reaches 9/33 and 17/33. Under the same local wrapper and MiniMax model, CAI reaches 2/33 flags and 4/33 useful findings, while VulnBot reaches 5/33 and 6/33. The blind E2E rate matches the published GPT-4o baseline, while the gap with purpose-built systems is expected: our harness is designed for multi-hop IoT networks, not single-host exploit chaining.

Vulhub. Vulhub has reproducible vulnerable environments but no uniform flag oracle, so we report useful-finding lower bounds. Table 6 separates published context from our controlled 328-case snapshot. Published rows are not directly comparable because target subsets, models, and oracles differ; our lower block uses the same MiniMax-M2.7 model, local Docker harness, and useful-finding oracle.

On Vulhub, informed/blind LANCE produce 99 and 92 useful findings, while CAI and VulnBot produce 170 and 92. These are single-host transfer diagnostics, not replacements for the IoTChainBench result: CAI is stronger on this oracle, while LANCE is the only evaluated system designed to reason over multi-device topology in the main benchmark.

8. Discussion and Limitations

Scope of the comparison. The controlled IoTChainBench comparison fixes the model and evaluator across systems. The adapted baselines are executed once per target device and their findings are merged at scenario level; LANCE instead evaluates the scenario as a network. The reported gap should therefore be read as evidence that network-level state matters in these IoT scenarios, not as a claim about stronger single-host exploitation.

Scope of transfer results. The AutoPenBench and Vulhub runs test the single-target exploitation component under

TABLE 6. VULHUB TRANSFER RESULTS. PUBLISHED ROWS PROVIDE CONTEXT; THE LOWER BLOCK IS OUR CONTROLLED 328-CASE COMPARISON UNDER THE SAME USEFUL-FINDING ORACLE.

System	Model	Oracle	Cases	Result
Published results (target selection and oracle differ)				
PentestAgent [11]	n/r	full-process success	67	74.2%
CheckMate [15]	Sonnet 4.5 + planner	shell access (M7)	120	88%
This work — same 328-case Vulhub snapshot and oracle				
LANCE informed	MiniMax-M2.7	useful findings	328	99/328 (30.2%)
LANCE blind	MiniMax-M2.7	useful findings	328	92/328 (28.0%)
CAI adapter [12]	MiniMax-M2.7	useful findings	328	170/328 (51.8%)
VulnBot adapter [9]	MiniMax-M2.7	useful findings	328	92/328 (28.0%)

PentestAgent and CheckMate use undisclosed Vulhub subsets. Lower block: denominator is the full 328-case list; useful findings are confirmed exploits plus probable vulnerabilities with machine-readable evidence.

established single-host workloads. They do not exercise the topology prior, per-device decomposition, or lateral-movement phases that define the network-native setting. These results show how the single-target component behaves outside IoTChainBench; the main architectural comparison remains the multi-device IoTChainBench evaluation.

Limitations. The following limitations bound the generalisability of our results. (i) *Simulated infrastructure.* IoTChainBench scenarios emulate IoT devices on Linux containers; real constrained hardware introduces embedded-specific attack surfaces (bootloaders, RTOS, firmware) not covered here. (ii) *Run-to-run variance.* We include a same-artifact rerun sanity check in §7.2, but do not estimate confidence intervals over repeated API calls. (iii) *Evaluator leniency.* Bonus categories are accepted as non-FP without ground-truth matches, which may inflate precision on scenarios with broad bonus lists. (iv) *Prompt sensitivity.* A single prompt family is used across all models; per-model tuning may change absolute scores. (v) *Static topologies.* The five architectural patterns are fixed; real deployments include devices not represented here (additional PLC families, medical devices, RFID readers) and evolve over time. (vi) *Ground-truth injection scope.* IoTChainBench ground truth reflects vulnerabilities that are tractably injectable via Ansible; harder-to-automate weaknesses (physical-layer timing attacks, supply-chain firmware backdoors) are not represented. Scenarios may therefore underestimate the difficulty of real IoT audits.

9. Ethical Considerations

All experiments run on an isolated Proxmox cluster with no route to external networks. Benchmark agents do not probe live hosts, production services, or third-party infrastructure.

All vulnerabilities are deterministically injected into our own lab containers from public vulnerability classes and documented misconfiguration patterns. The work does not introduce or disclose new third-party vulnerabilities; no vendor notification process is triggered. External corpora are run locally from their public artifacts and used only for measurement.

The released artifact contains deployment playbooks, ground truth, evaluator code, and run logs. It includes no private targets, credentials, or novel exploit payloads, and is intended for security research and benchmark reproduction in controlled lab environments.

10. Conclusion

LLM pentest agents are often evaluated on isolated hosts, but IoT compromises depend on reachability across devices, protocol roles, and pivot-dependent findings. This paper introduced IoTChainBench, a reproducible network-scale benchmark with 13 vulnerability-injected IoT/OT scenarios, two hardened controls, 145 devices total, and 229 injected vulnerabilities, together with LANCE, a six-phase network-native harness. IoTChainBench adds per-vulnerability ground truth with severity, verification commands, and reachability metadata for pivot-aware analysis.

In the controlled comparison, LANCE achieves 0.940 mean F1 and an 86.1% CVSS-weighted score on the 10 main scenarios, outperforming each adapted baseline under the same model and evaluator. The result supports the main claim of the paper: evaluating LLM pentest agents for IoT requires benchmarks that expose network structure, not only single-host exploit completion. All artifacts are released at `ANONYMIZED_URL`.

References

[1] M. Antonakakis *et al.*, "Understanding the Mirai botnet," in *Proc. 26th USENIX Security Symp.*, 2017.

[2] W. Turton, "Hackers breach thousands of security cameras, exposing Tesla, jails, hospitals," *Bloomberg*, Mar. 2021.

[3] R. Langner, "Stuxnet: Dissecting a cyberwarfare weapon," *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.

[4] A. Happe and J. Cito, "Getting pwn'd by AI: Penetration testing with large language models," in *Proc. ACM ESEC/FSE*, 2023.

[5] G. Deng *et al.*, "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *Proc. 33rd USENIX Security Symp.*, 2024.

[6] A. Happe *et al.*, "HackingBuddyGPT: An LLM-driven pentest agent," 2024. [Online]. Available: https://github.com/ipa-lab/hackingBuddyGPT

[7] J. Xu *et al.*, "AutoAttacker: A large language model guided system to implement automatic cyber-attacks," *arXiv:2403.01038*, 2024.

[8] O. V. Aguinaga, "Nebula: An LLM-powered penetration testing assistant," *arXiv:2412.00006*, 2024.

[9] H. He *et al.*, "VulnBot: Autonomous penetration testing for a multi-agent collaborative framework," *arXiv:2501.13411*, 2025.

[10] Y. Ginige *et al.*, "AutoPentester: An LLM agent-based framework for automated pentesting," *arXiv:2510.05605*, 2025.

[11] X. Shen *et al.*, "PentestAgent: Incorporating LLM agents to automated penetration testing," in *Proc. 20th ACM Asia Conf. Computer and Communications Security (ASIA CCS)*, 2025. doi: 10.1145/3708821.3733882.

[12] V. Mayoral-Vilches *et al.*, "CAI: An open, bug bounty-ready cybersecurity AI," *arXiv:2504.06017*, 2025.

[13] Anonymous, "Pentest-R1: Towards autonomous penetration testing reasoning optimized via two-stage reinforcement learning," *arXiv:2508.07382*, 2025.

[14] P. D. Luong *et al.*, "xOffense: An autonomous multi-agent framework for penetration testing with domain-adapted large language models," *arXiv:2509.13021*, 2025.

[15] L. Wang, X. Shi, Z. Li, Y. Jiang, S. Tan, Y. Jiang, J. Cheng, W. Chen, X. Shen, Z. Li, and Y. Chen, "Automated penetration testing with LLM agents and classical planning," *arXiv:2512.11143*, 2025.

[16] L. Gioacchini, M. Mellia, I. Drago, A. Delsanto, G. Siracusano, and R. Bifulco, "AutoPenBench: Benchmarking generative agents for penetration testing," *arXiv:2410.03225*, 2024.

[17] Vulhub contributors, "Vulhub: Pre-built vulnerable Docker environments for learning and reproducing CVEs." [Online]. Available: https://github.com/vulhub/vulhub

[18] V. Mayoral-Vilches *et al.*, "CAIBench: A meta-benchmark for evaluating cybersecurity AI agents," *arXiv:2510.24317*, 2025.

[19] M. Shao *et al.*, "NYU CTF Bench: A scalable open-source benchmark dataset for evaluating LLMs in offensive security," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2024.

[20] A. K. Zhang *et al.*, "Cybench: A framework for evaluating cybersecurity capabilities and risk of language models," *arXiv:2408.08926*, 2024.

[21] Ludus Cloud, "Ludus: Cyber range automation on Proxmox." [Online]. Available: https://ludus.cloud

[22] M3, "GOAD: Game of Active Directory." [Online]. Available: https://github.com/Orange-Cyberdefense/GOAD

[23] C. Phillips and L. P. Swiler, "A graph-based system for network-vulnerability analysis," in *Proc. New Security Paradigms Workshop*, 1998.

[24] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proc. IEEE Symp. Security and Privacy*, 2002.

[25] X. Ou, S. Govindavajhala, and A. W. Appel, "MulVAL: A logic-based network security analyzer," in *Proc. USENIX Security Symp.*, 2005.

[26] OWASP Foundation, "OWASP Internet of Things Top 10," 2024. [Online]. Available: https://owasp.org/www-project-internet-of-things-top-10/

[27] MITRE Corporation, "MITRE ATT&CK for ICS," 2024. [Online]. Available: https://attack.mitre.org/matrices/ics/

[28] K. Stouffer *et al.*, "Guide to operational technology security," National Institute of Standards and Technology, NIST SP 800-82 Rev. 3, 2023.

[29] S. Garcia, A. Parmisano, and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," Stratosphere Laboratory, 2020. [Online]. Available: https://www.stratosphereips.org/datasets-iot23

[30] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165130–165150, 2020.

[31] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022.

[32] A. Costin, J. Zaddach, A. Francillon, and D. Balzarotti, "A large-scale analysis of the security of embedded firmwares," in *Proc. USENIX Security Symp.*, 2014.

[33] D. D. Chen, M. Egele, M. Woo, and D. Brumley, "Towards automated dynamic analysis for Linux-based embedded firmware," in *Proc. NDSS*, 2016.

[34] M. Kim, D. Kim, E. Kim, S. Kim, Y. Jang, and Y. Kim, "FirmAE: Towards large-scale emulation of IoT firmware for dynamic analysis," in *Proc. Annual Computer Security Applications Conf. (ACSAC)*, 2020.

[35] OWASP, "IoTGoat: A deliberately insecure IoT firmware," 2024. [Online]. Available: https://github.com/OWASP/IoTGoat

[36] T. Abramovich *et al.*, "EnIGMA: Interactive tools substantially assist LM agents in solving CTF challenges," in *Proc. ICML*, 2025.

[37] MiniMax, "MiniMax-M2.7: A general-purpose large language model," 2026. [Online]. Available: https://www.minimax.io/models/text/m27

LLM Usage Statement

LLMs were used for editorial assistance while preparing this manuscript, and all generated text was inspected by the authors to ensure accuracy, originality, and consistency with the underlying artifacts. LLMs are also part of the studied methodology: LANCE uses tool-calling LLM agents for network reconnaissance, vulnerability analysis, exploit verification, intrusion attempts, and report generation, as described in §5, §6, and §7.